Memory Safety Vulnerabilities Part 2

CS 161 Spring 2026

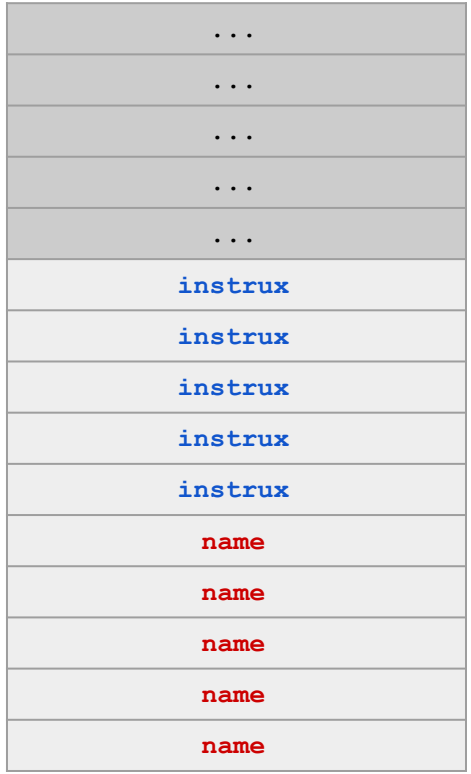
Reminder: buffer overflow


```
char name[20];
char instrux[20] = "none";

void vulnerable(void) {
    ...
    gets(name);
    ...
}
```

gets starts writing here and can overwrite anything above **name**!

Note: **name** and **instrux** are declared in static memory (outside of the stack), which is why **name** is below **instrux**



Reminder: Integer overflow


```
void func(int len, char *data) {
    char buf[64];
    if (len > 64)
        return;
    memcpy(buf, data, len);
}
```

This is a **signed** comparison, so `len > 64` will be false. But when we call memcpy(), casting **-1** to an unsigned type yields `0xffffffff`: another buffer overflow!

```
void *memcpy(void *dest, const void *src, size_t n);
```

Format String Vulnerabilities

Textbook Chapter 3.3

`printf` behavior

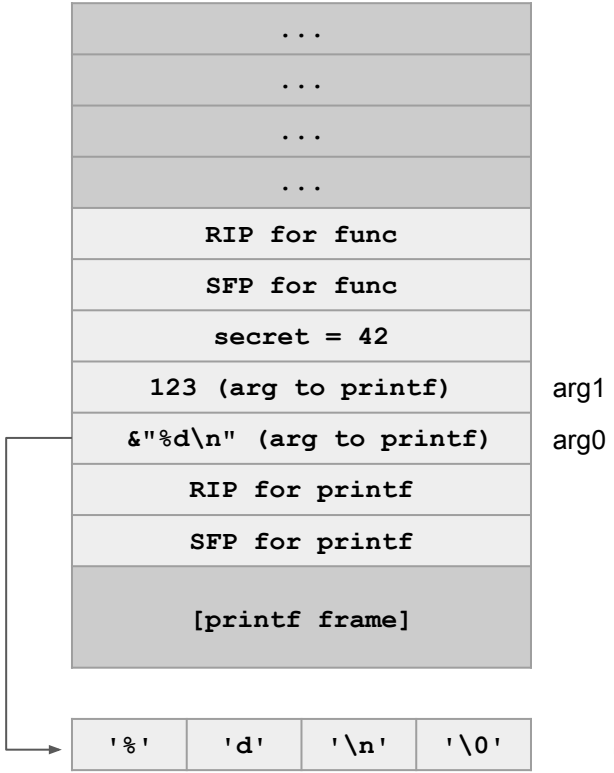
- `printf` accepts a variable number of arguments
  - How does it know how many arguments that it received?
  - It infers it from the first argument: the format string!
  - Example: `printf("One %s costs %d", fruit, price)`
  - What happens if the arguments are mismatched?

`printf` behavior


```
void func(void) {
    int secret = 42;
    printf("%d\n", 123);
}
```

`printf` assumes that there is 1 more argument because there is one format specifier (%d), so it will look 4 bytes up the stack for the argument

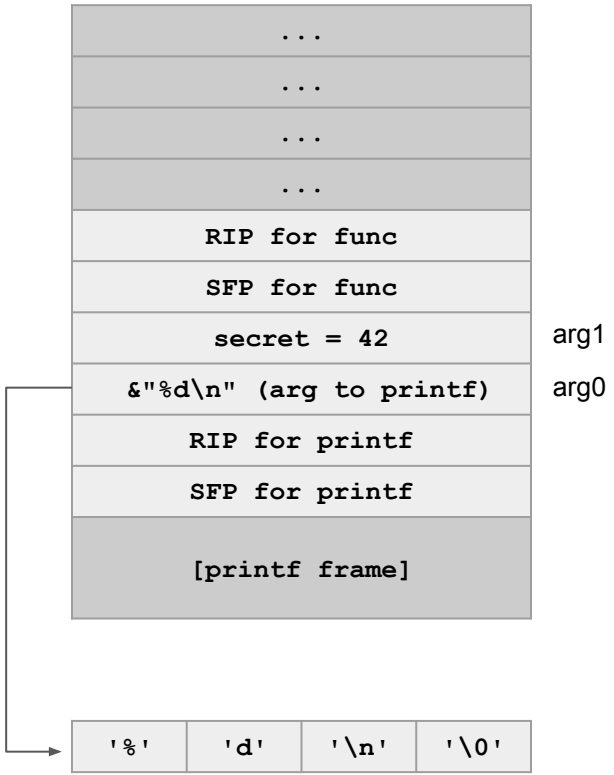
What if there is no argument?



`printf` behavior


```
void func(void) {
    int secret = 42;
    printf("%d\n");
}
```

Because the format string contains the `%d`, it will still look 4 bytes up -- and print the value of `secret`!



Format String Vulnerabilities

What is the issue here?

```
char buf[64];

void vulnerable(void) {
    if (fgets(buf, 64, stdin) == NULL)
        return;
    printf(buf);
}
```

Format String Vulnerabilities

- With this code, the attacker can specify any format string they want:
  - `printf("100% done!")`
    - Prints 4 bytes on the stack, 8 bytes above the RIP of `printf`
  - `printf("100% stopped.")`
    - Print the bytes **pointed to** by the address located 8 bytes above the RIP of `printf`, until the first '\0' byte
  - `printf("%x %x %x %x ...")`
    - Print a series of values on the stack in hex

```
char buf[64];

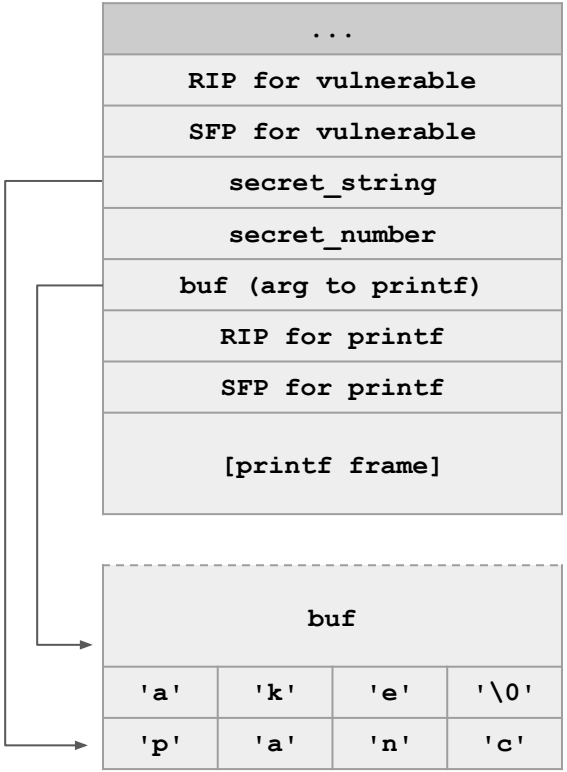
void vulnerable(void) {
    if (fgets(buf, 64, stdin) == NULL)
        return;
    printf(buf);
}
```

Format String Vulnerability Walkthrough


```
char buf[64];

void vulnerable(void) {
    char *secret_string = "pancake";
    int secret_number = 42;
    if (fgets(buf, 64, stdin) == NULL)
        return;
    printf(buf);
}
```

Note that strings are passed by reference in C, so the argument to **`printf`** is actually a pointer to **`buf`**, which is in static memory.



Format String Vulnerability Walkthrough

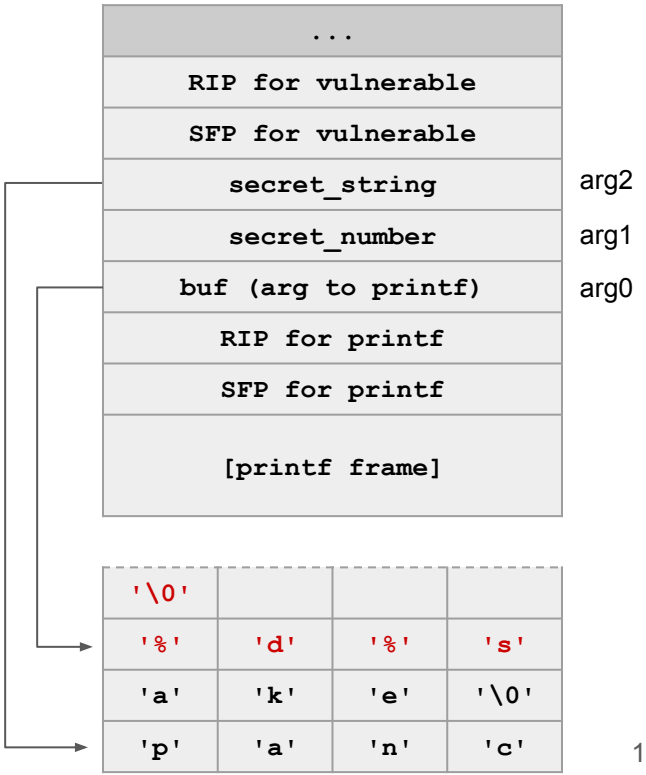
Input: `%d%s`

Output:

```
char buf[64];

void vulnerable(void) {
    char *secret_string = "pancake";
    int secret_number = 42;
    if (fgets(buf, 64, stdin) == NULL)
        return;
    printf(buf);
}
```

We're calling **`printf("%d%s")`**. **`printf`** reads its first argument (arg0), sees two format specifiers, and expects two more arguments (arg1 and arg2).



Format String Vulnerability Walkthrough

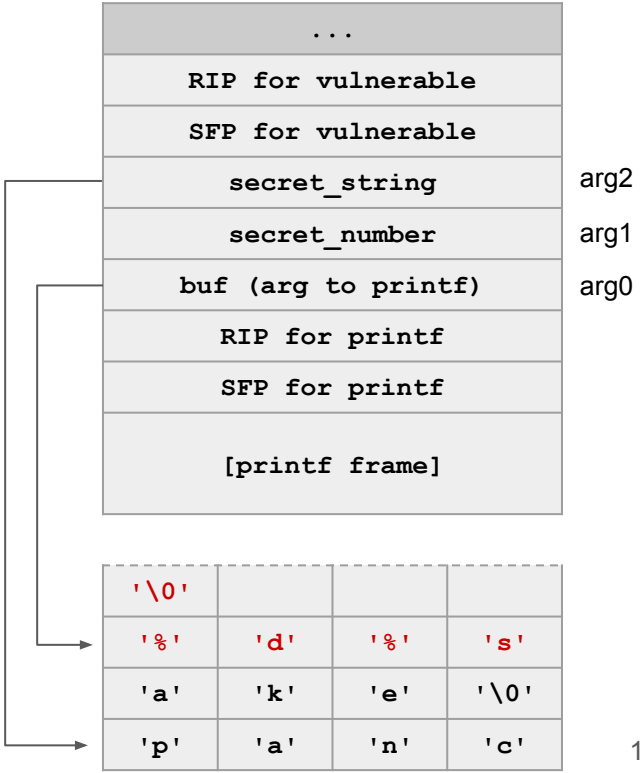
Input: `%d%s`

Output:
42

```
char buf[64];

void vulnerable(void) {
    char *secret_string = "pancake";
    int secret_number = 42;
    if (fgets(buf, 64, stdin) == NULL)
        return;
    printf(buf);
}
```

The first format specifier `%d` says to treat the next argument (arg1) as an integer and print it out.



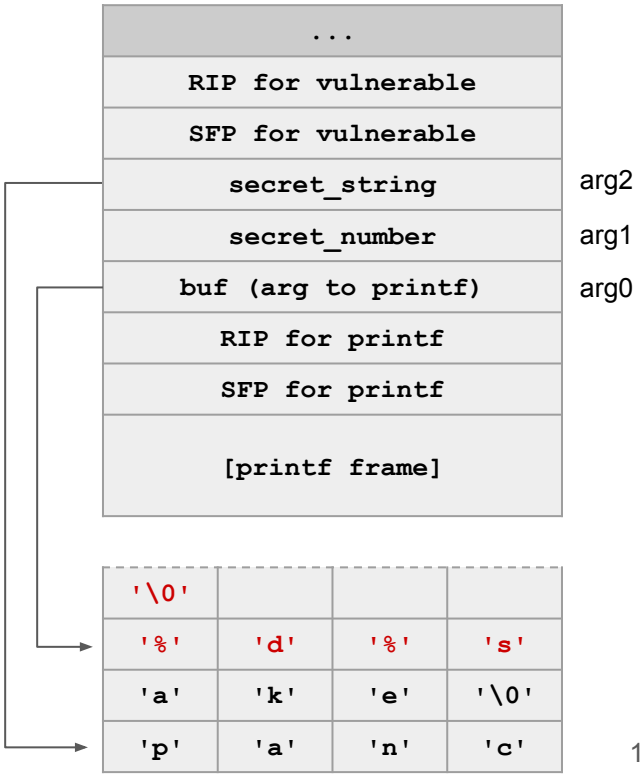
Format String Vulnerability Walkthrough

Input: `%d%s`

Output: **42pancake**

```
char buf[64];

void vulnerable(void) {
    char *secret_string = "pancake";
    int secret_number = 42;
    if (fgets(buf, 64, stdin) == NULL)
        return;
    printf(buf);
}
```



The second format specifier `%s` says to treat the next argument (arg2) as a (pointer to a) string and

`%s` will dereference the pointer at arg2 and print until it sees a null byte (`'\0'`)

Format String Vulnerabilities

- **printf** can also write values using the `%n` specifier
  - `%n` treats the next argument as a **pointer** and writes the number of bytes printed so far to that address (usually used to calculate output spacing)
    - `printf("item %d:%n", 3, &val)` stores 7 in `val`
    - `printf("item %d:%n", 987, &val)` stores 9 in `val`
  - `printf("000%n")`
    - **Writes** the value 3 to the memory location **pointed to** by address located 8 bytes above the RIP of `printf`

```
void vulnerable(void) {
    char buf[64];
    if (fgets(buf, 64, stdin) == NULL)
        return;
    printf(buf);
}
```

Format String Vulnerability Walkthrough

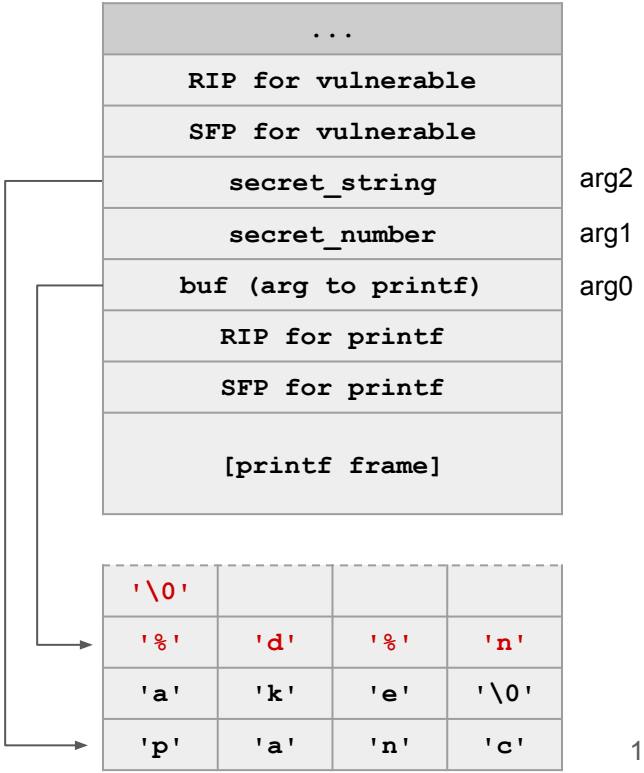
Input: `%d%n`

Output:

```
char buf[64];

void vulnerable(void) {
    char *secret_string = "pancake";
    int secret_number = 42;
    if (fgets(buf, 64, stdin) == NULL)
        return;
    printf(buf);
}
```

We're calling **`printf("%d%n")`**. **`printf`** reads its first argument (arg0), sees two format specifiers, and expects two more arguments (arg1 and arg2).



Format String Vulnerability Walkthrough

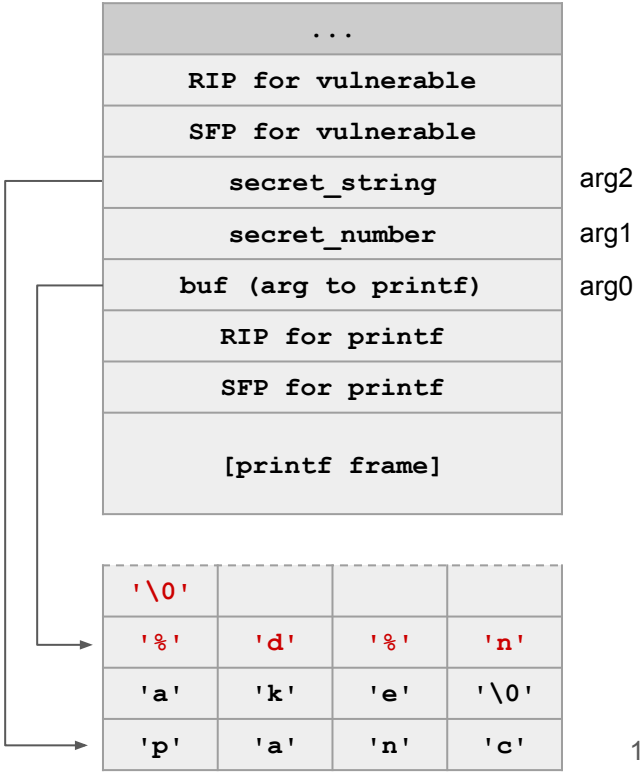
Input: `%d%n`

Output:
42

```
char buf[64];

void vulnerable(void) {
    char *secret_string = "pancake";
    int secret_number = 42;
    if (fgets(buf, 64, stdin) == NULL)
        return;
    printf(buf);
}
```

The first format specifier `%d` says to treat the next argument (arg1) as an integer and print it out.



Format String Vulnerability Walkthrough

Input: `%d%n`

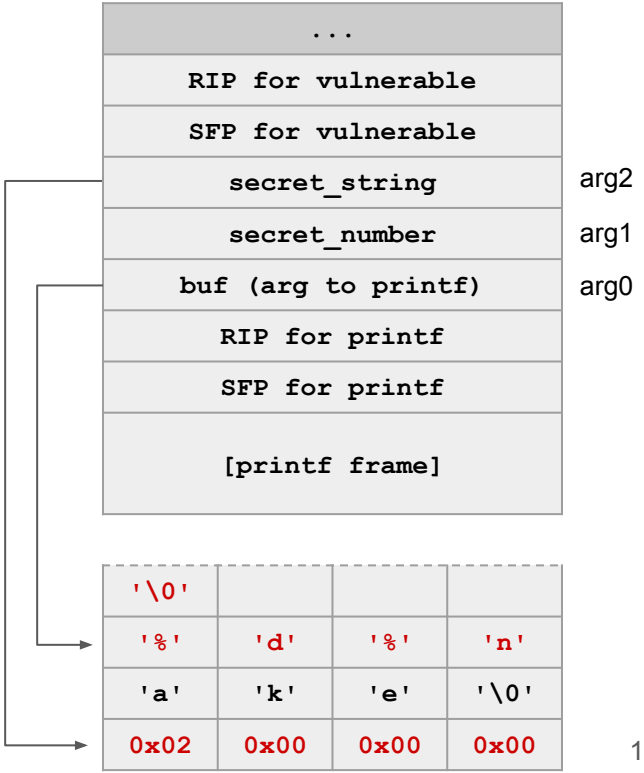
Output:
42

```
char buf[64];

void vulnerable(void) {
    char *secret_string = "pancake";
    int secret_number = 42;
    if (fgets(buf, 64, stdin) == NULL)
        return;
    printf(buf);
}
```

The second format specifier `%n` says to treat the next argument (arg2) as a pointer, and write the number of bytes printed so far to the address at arg2.

We've printed 2 bytes so far, so the number 2 gets written to **`secret_string`**.



Format Strings: Stack Diagram


```
void vulnerable(void) {
    char buf[16];
    char str[12];
    fgets(buf, 16, stdin);
    printf(buf);
}
```

Now, let's try some format string vulnerabilities where the user-controlled buffer is on the stack instead of in static memory.

What does the stack diagram look like?

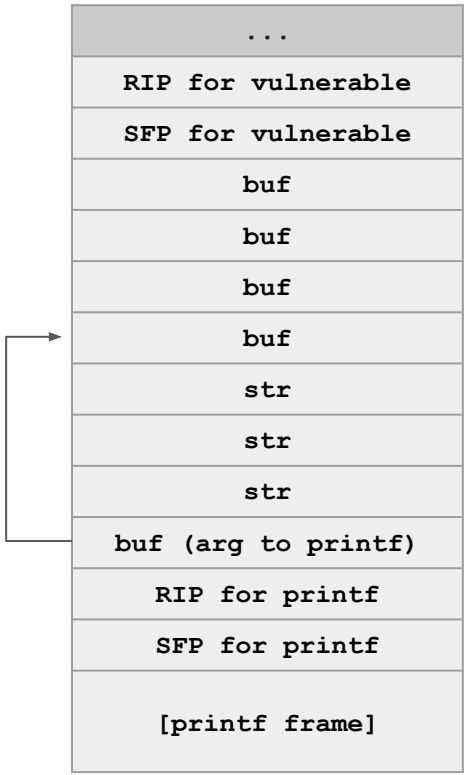


Format Strings: Stack Diagram


```
void vulnerable(void) {
    char buf[16];
    char str[12];
    fgets(buf, 16, stdin);
    printf(buf);
}
```

This is the stack diagram while **`printf`** is being called.

Where does **`printf`** look for arguments?

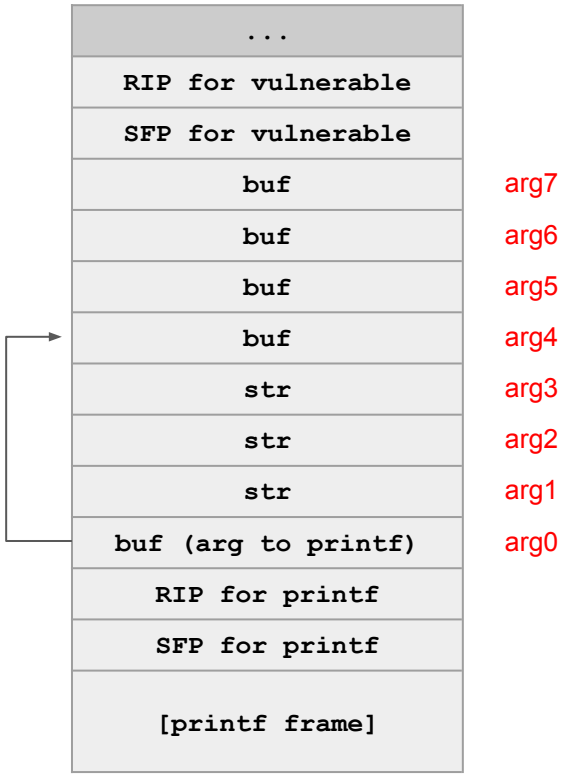


Format Strings: Stack Diagram


```
void vulnerable(void) {
    char buf[16];
    char str[12];
    fgets(buf, 16, stdin);
    printf(buf);
}
```

We've labeled which values in memory **`printf`** will interpret as arguments.

For example, if **`buf`** has 4 percent formatters, **`printf`** will match the last percent formatter with arg4.

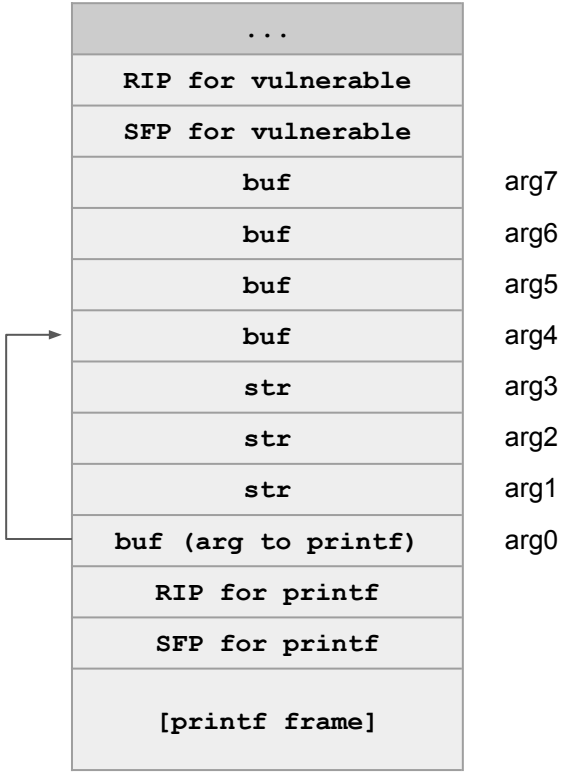


Write 100 to `0xdeadbeef`

```
void vulnerable(void) {
      char buf[16];
      char str[12];
      fgets(buf, 16, stdin);
      printf(buf);
}
```


Attack scenario: Write the number 100 to memory address `0xdeadbeef`.

What input should the attacker supply?



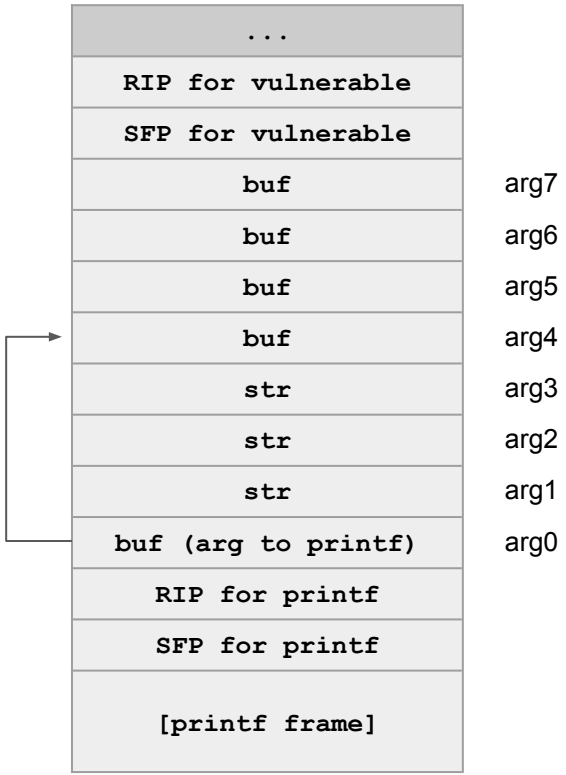
Write 100 to `0xdeadbeef`

```
void vulnerable(void) {
        char buf[16];
        char str[12];
        fgets(buf, 16, stdin);
        printf(buf);
}
```


Recall: When **`printf`** sees a **`%n`**, it takes the next unused argument, treats it like an address, and writes *the number of bytes printed so far* to that address.

When **`printf`** sees the **`%n`**, two things need to be true:

- Control *where* we write: The next unused argument on the stack should be **`0xdeadbeef`**.
- Control *what* we write: *The number of bytes printed so far* should be 100.



Write 100 to `0xdeadbeef`

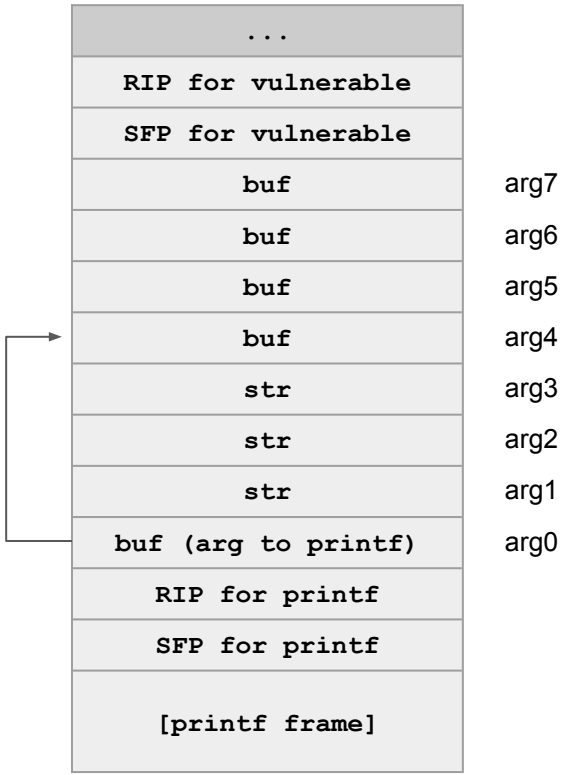
```
void vulnerable(void) {
        char buf[16];
        char str[12];
        fgets(buf, 16, stdin);
        printf(buf);
}
```


When **`printf`** sees the **`%n`**, two things need to be true:

- Control *where* we write: The next unused argument on the stack should be **`0xdeadbeef`**.
- Control *what* we write: *The number of bytes printed so far* should be 100.

Consider this exploit. What does it look like in memory?

Input:	`0xdeadbeef`	`%94c`	`%c`	`%c`	`%n`



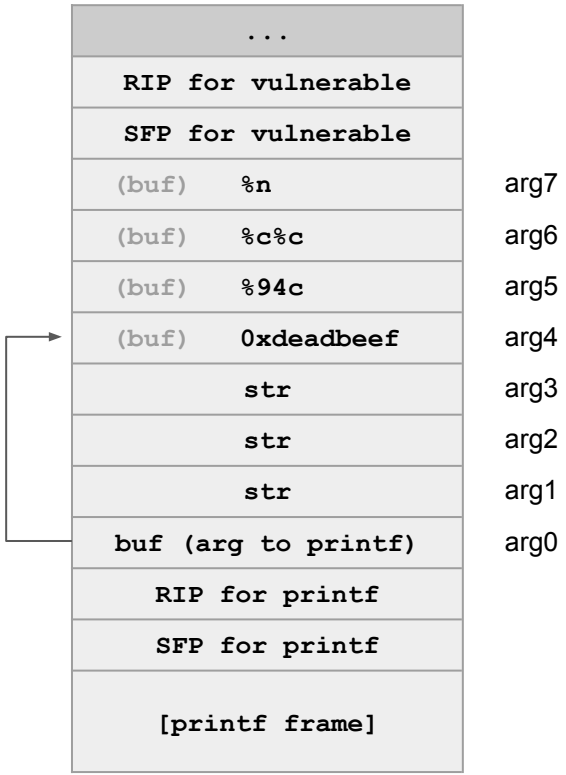
Write 100 to `0xdeadbeef`

```
void vulnerable(void) {
        char buf[16];
        char str[12];
        fgets(buf, 16, stdin);
        printf(buf);
}
```


When writing to memory, the percent formatters take up multiple bytes of memory.

For example, `%94c` is 4 characters and takes up 4 bytes of memory.

Input:	`0xdeadbeef`	`%94c`	`%c`	`%c`	`%n`
# chars used:	4	4	2	2	2



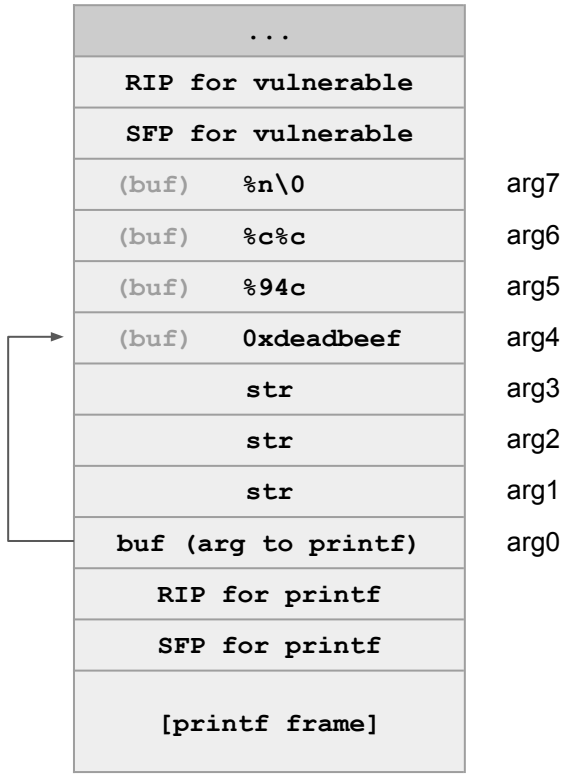
Write 100 to `0xdeadbeef`

```
void vulnerable(void) {
        char buf[16];
        char str[12];
        fgets(buf, 16, stdin);
        printf(buf);
}
```


Control *where* we write: The next unused argument on the stack should be `0xdeadbeef`.

- Each percent formatter "uses up" or "consumes" one argument on the stack.
- We added `%c` arguments to "consume" or "skip past" `str`, so that the `%n` argument aligns with arg4, where we put `0xdeadbeef`.

Input:	`0xdeadbeef`	`%94c`	`%c`	`%c`	`%n`
# chars used:	4	4	2	2	2
Consumes:	N/A	arg1	arg2	arg3	arg4



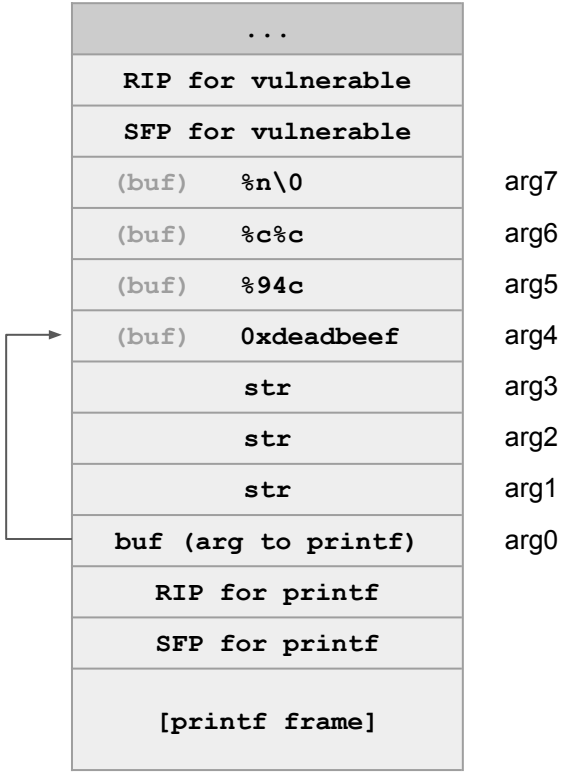
Write 100 to `0xdeadbeef`


```
void vulnerable(void) {
        char buf[16];
        char str[12];
        fgets(buf, 16, stdin);
        printf(buf);
}
```

Control *what* we write: *The number of bytes printed so far* should be 100.

- `%94c` prints the next argument on the stack as a character, padded to 94 bytes. (Also works if you switch 94 with other numbers.)
- `0xdeadbeef` and the `%c` formatters also caused characters to be printed, so we needed 100–4–1–1 = 94 padding bytes.

Input:	`0xdeadbeef`	`%94c`	`%c`	`%c`	`%n`
# chars used:	4	4	2	2	2
Consumes:	N/A	arg1	arg2	arg3	arg4
# bytes printed:	4	94	1	1	0



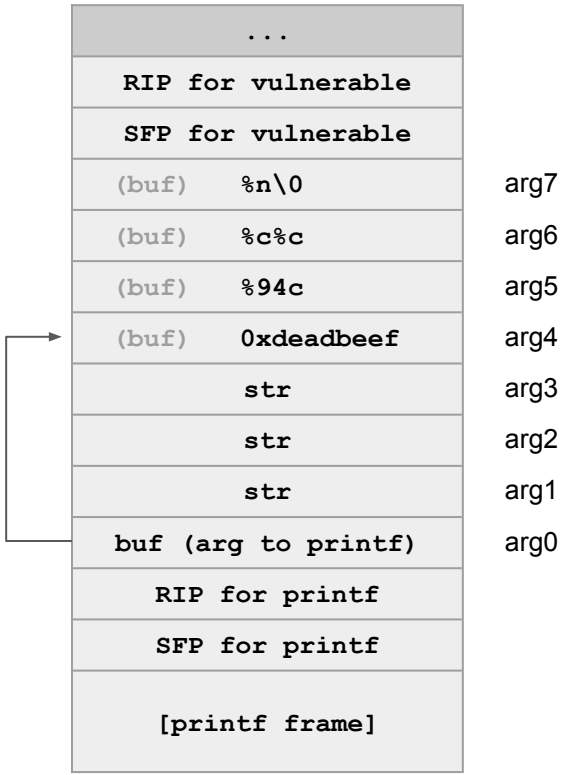
Write 100 to `0xdeadbeef`


```
void vulnerable(void) {
        char buf[16];
        char str[12];
        fgets(buf, 16, stdin);
        printf(buf);
}
```

How would you modify this exploit to write to address **`0xbfff1234`** instead of **`0xdeadbeef`**?

How would you modify this exploit to write 89 to **`0xbfff1234`** instead of writing 100?

Input:	`0xdeadbeef`	`%94c`	`%c`	`%c`	`%n`
# chars used:	4	4	2	2	2
Consumes:	N/A	arg1	arg2	arg3	arg4
# bytes printed:	4	94	1	1	0



Format String Vulnerabilities: Defense


```
void vulnerable(void) {
    char buf[64];
    if (fgets(buf, 64, stdin) == NULL)
        return;
    printf("%s", buf);
}
```

Never use untrusted input in the first argument to **`printf`**.

Now the attacker can't make the number of arguments mismatched!

Implications

- If the code has a format string vulnerability, attacker can write any byte value they want, to any address in memory they want
  - Can also repeat multiple times
- This is enough to overwrite a RIP, overwrite a function pointer, etc.
- => Enables attacker to trigger execution of arbitrary malicious shellcode

Heap Vulnerabilities

Textbook Chapter 3.6

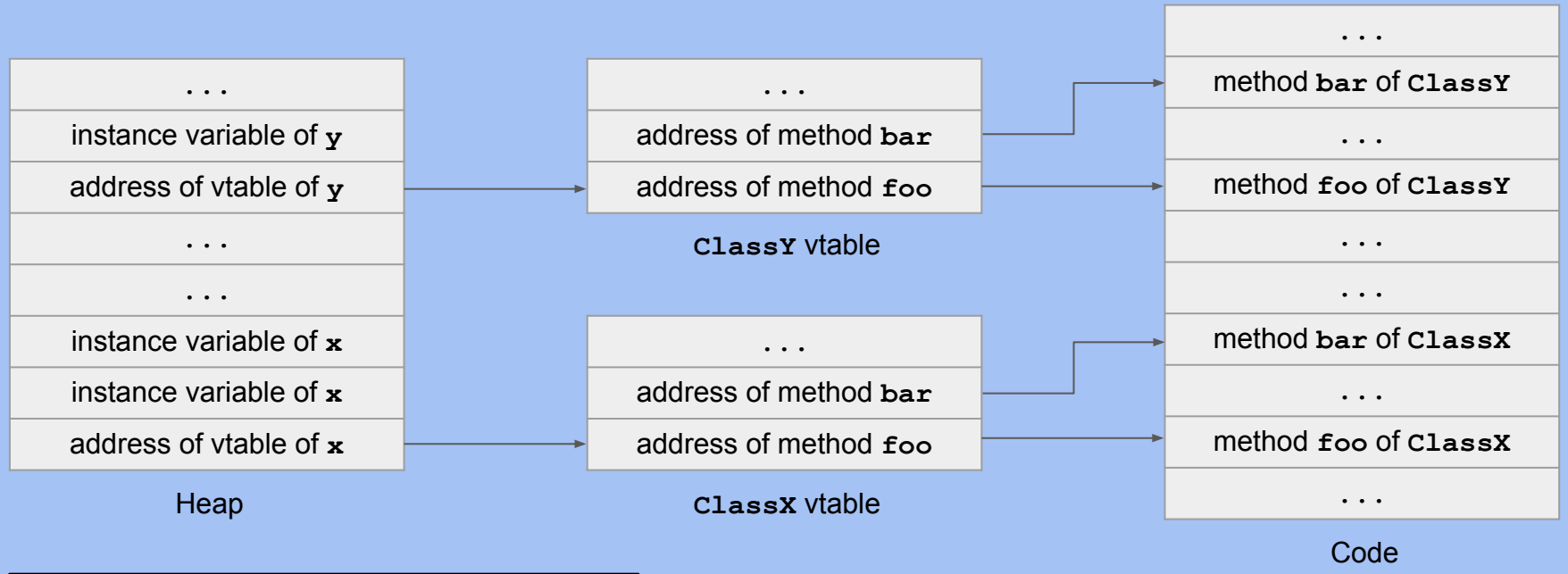
Targeting Instruction Pointers

- Remember: You need to overwrite a pointer that will eventually be jumped to
- Stack smashing involves the RIP, but there are other targets too (literal function pointers, etc.)

C++ vtables

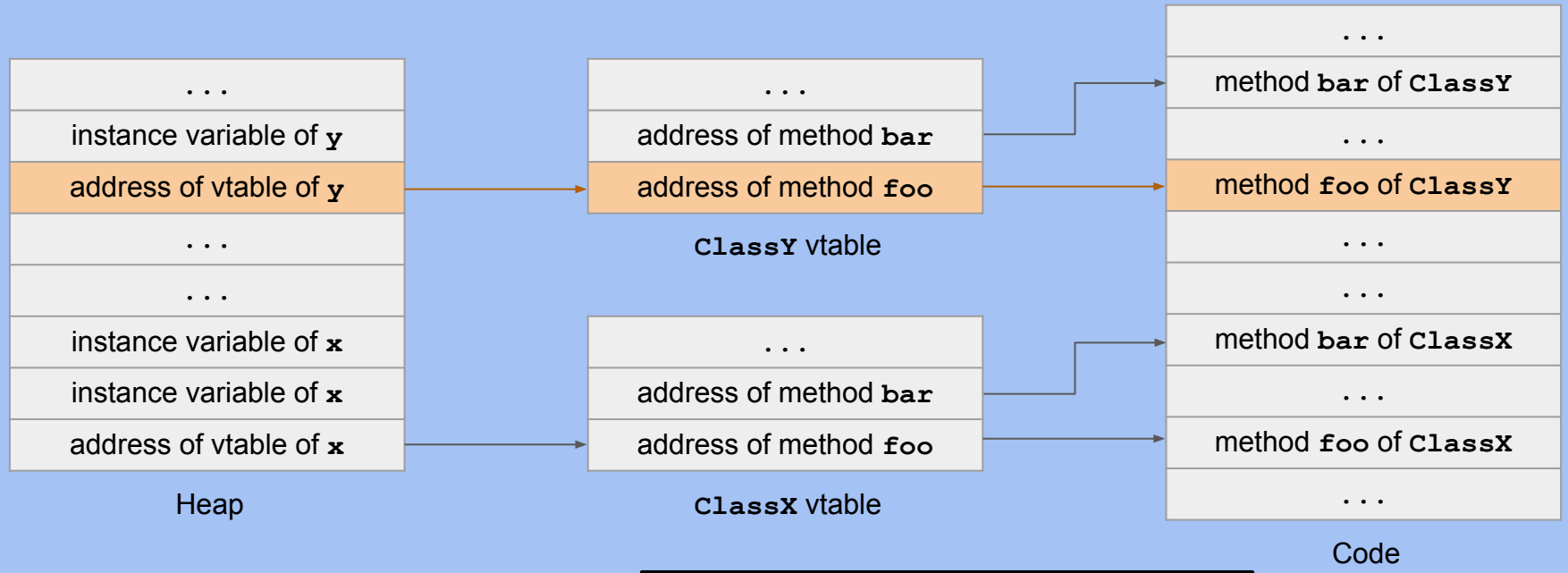
- C++ is an object-oriented language
  - C++ objects can have instance variables and methods
  - C++ has *polymorphism*: implementations of an interface can implement functions differently, similar to Java
- To achieve this, each class has a vtable (table of function pointers), and each object points to its class's vtable
  - The vtable pointer is usually at the beginning of the object
  - To execute a function: Dereference the vtable pointer with an offset to find the function address

C++ vtables



x is an object of type **ClassX**.
y is an object of type **ClassY**.

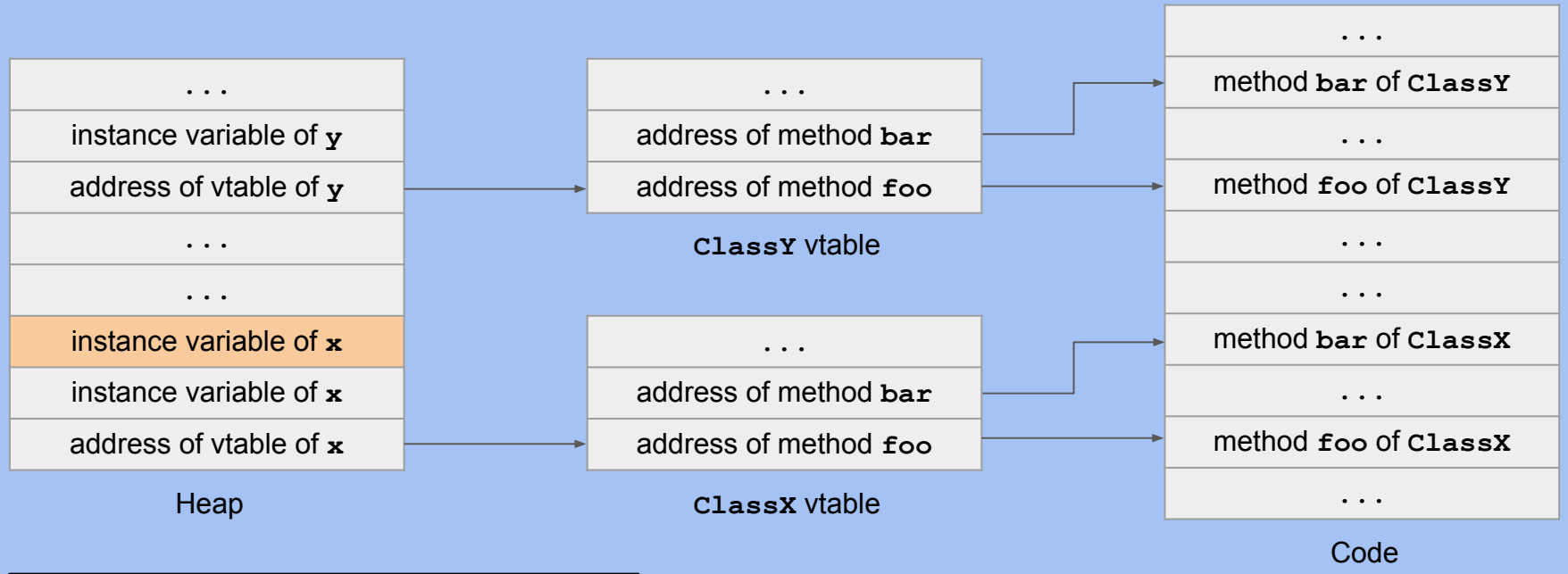
C++ vtables



To call a method of `y`, first follow a pointer on the heap to find the vtable…

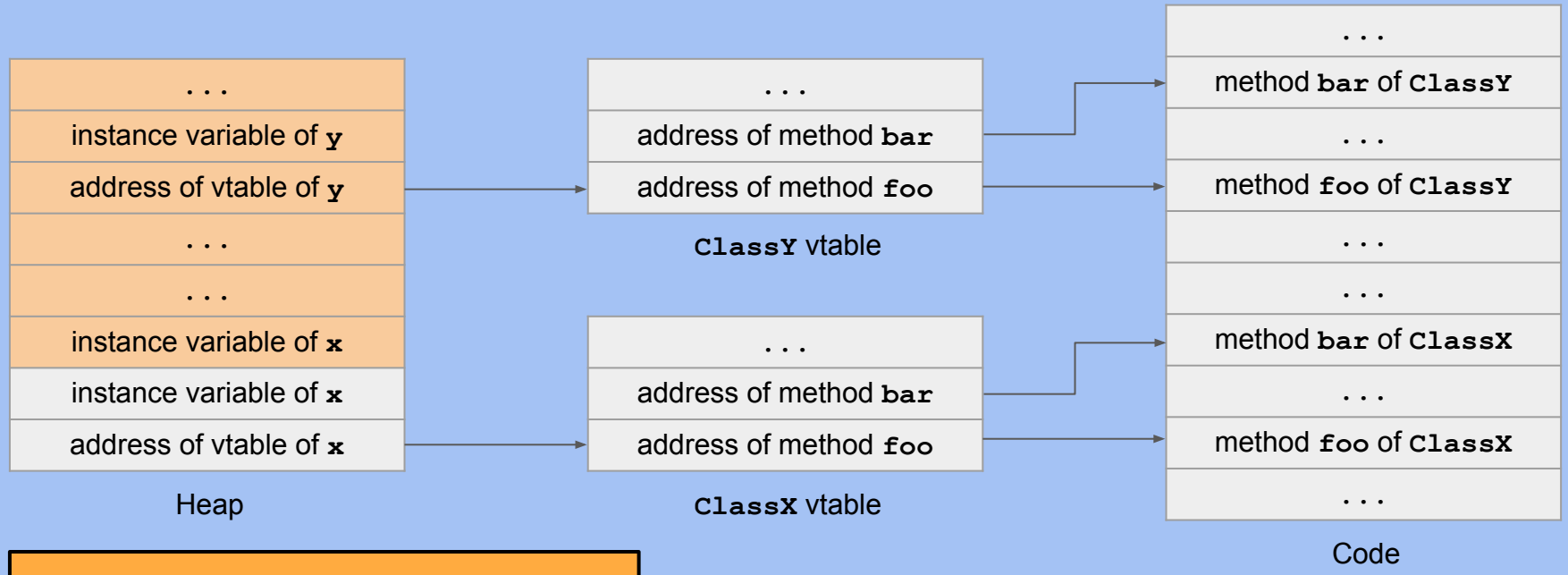
… then follow a pointer in the vtable to find the instructions of the method.

C++ vtables



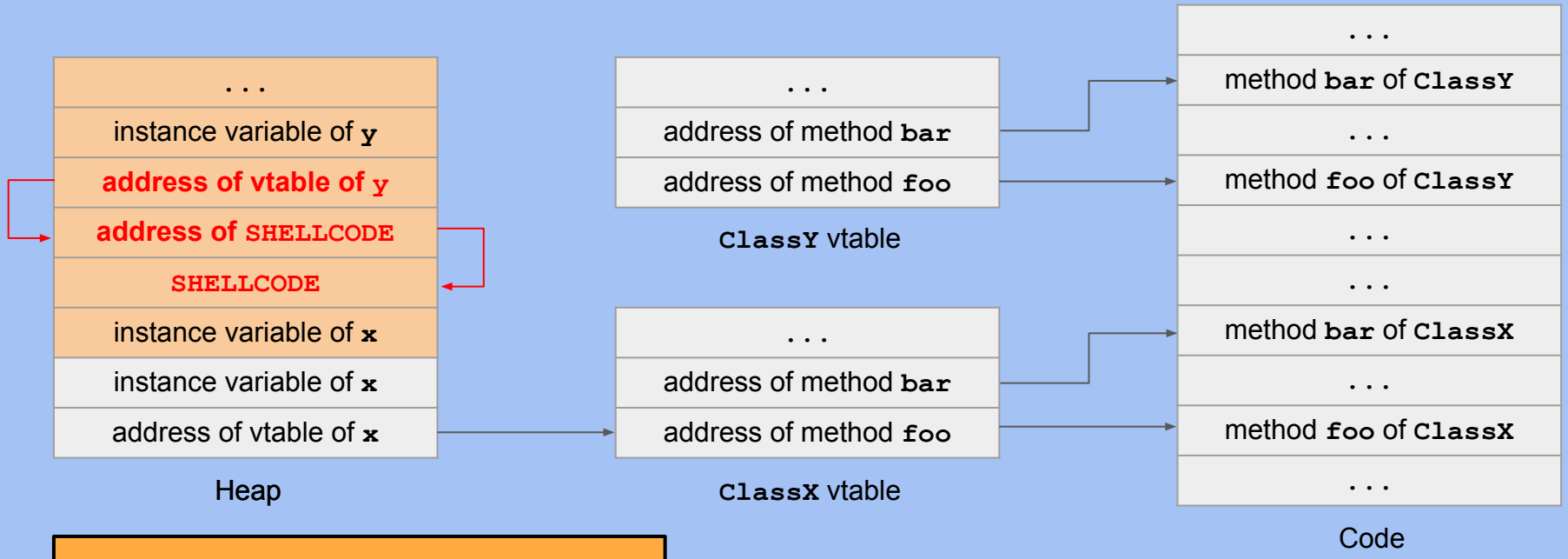
Suppose one of the instance variables of **x** is a buffer we can overflow.

C++ vtables



The attacker controls everything above the instance variable of **x** on the heap, including the vtable pointer for **y**.

C++ vtables



The vtable for `y` is now a pointer to shellcode. If method `foo` for `y` is called, it will execute shellcode!

Heap Vulnerabilities

- Heap overflow
  - Objects are allocated in the heap (using **`malloc`** in C or **`new`** in C++)
  - A write to a buffer in the heap is not checked
  - The attacker overflows the buffer and overwrites the vtable pointer of the next object to point to a malicious vtable, with pointers to malicious code
  - The next object's function is called, accessing the vtable pointer
- Use-after-free
  - An object is deallocated too early (using **`free`** in C or **`delete`** in C++)
  - The attacker allocates memory, which returns the memory freed by the object
  - The attacker overwrites a vtable pointer under the attacker's control to point to a malicious vtable, with pointers to malicious code
  - The deallocated object's function is called, accessing the vtable pointer

Top 10 Most Dangerous Software Weaknesses (2023)

Rank	ID	Name	Score
[1]	CWE-787	Out-of-bounds Write	63.72
[2]	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')	45.54
[3]	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')	34.27
[4]	CWE-416	Use After Free	16.71
[5]	CWE-78	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')	15.65
[6]	CWE-20	Improper Input Validation	15.50
[7]	CWE-125	Out-of-bounds Read	14.60
[8]	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')	14.11
[9]	CWE-352	Cross-Site Request Forgery (CSRF)	11.73
[10]	CWE-434	Unrestricted Upload of File with Dangerous Type	10.41



Self-study Material

Textbook Chapter 3.5

Off-by-One Exploit

Textbook Chapter 3.5

Off-by-one

Goal: Execute shellcode located at `0xdeadbeef`. What parts of memory is an attacker able to overwrite in this piece of code?

size_t **fread**(void *restrict ptr, size_t size, size_t nitems,
FILE *restrict stream);

The function **fread**() reads nitems objects, each size bytes long, from the stream pointed to by stream, storing them at the location given by ptr.

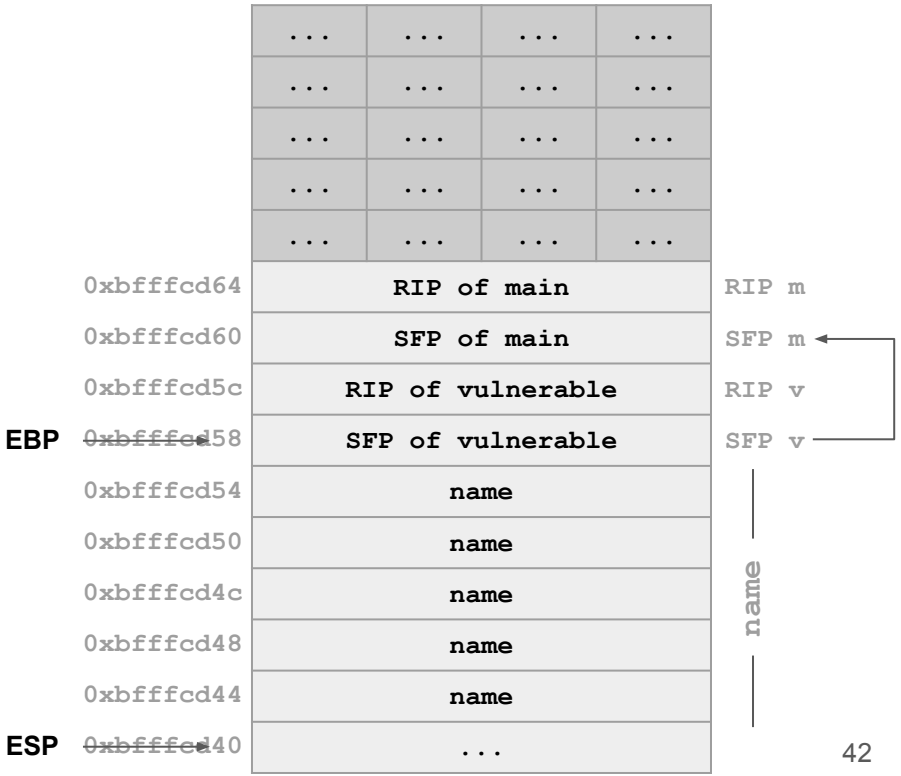
```
void vulnerable(void) {
    char name[20];
    fread(name,21,1,stdin);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call gets
    add $4, %esp
    mov %ebp, %esp
    pop %ebp
    ret

main:
    ...
    call vulnerable
    mov %ebp, %esp
    pop %ebp
    ret
```



Off-by-one

The attacker is able to overwrite all of **`name`** and the least-significant byte of the SFP of **`vulnerable`**.

If the attacker can change where the SFP of **`vulnerable`** points to, how can they use this to execute shellcode?

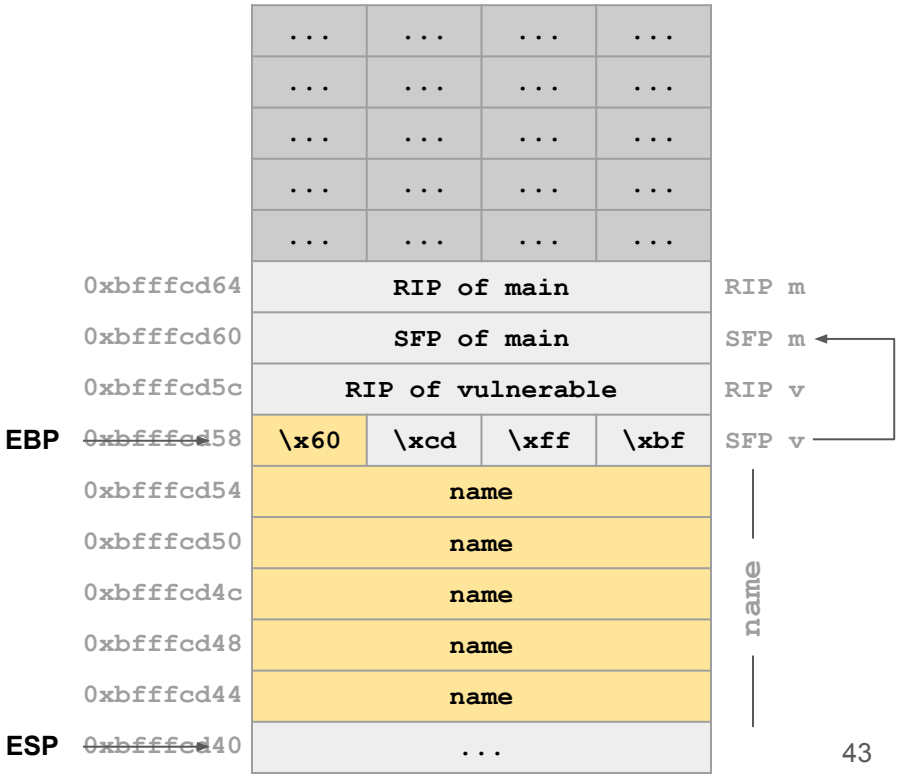
```
void vulnerable(void) {
    char name[20];
    fread(name,21,1,stdin);
}

int main(void) {
    vulnerable();
    return 0;
}
```

```
vulnerable:
    ...
    call gets
    add $4, %esp
    mov %ebp, %esp
    pop %ebp
    ret

main:
    ...
    call vulnerable
    mov %ebp, %esp
    pop %ebp
    ret
```

EIP →



Off-by-one

The SFP of **`vulnerable`** now points inside **`name`**, which the attacker controls.

What does the SFP usually point to? What will the C program interpret the first bytes of **`name`** as?

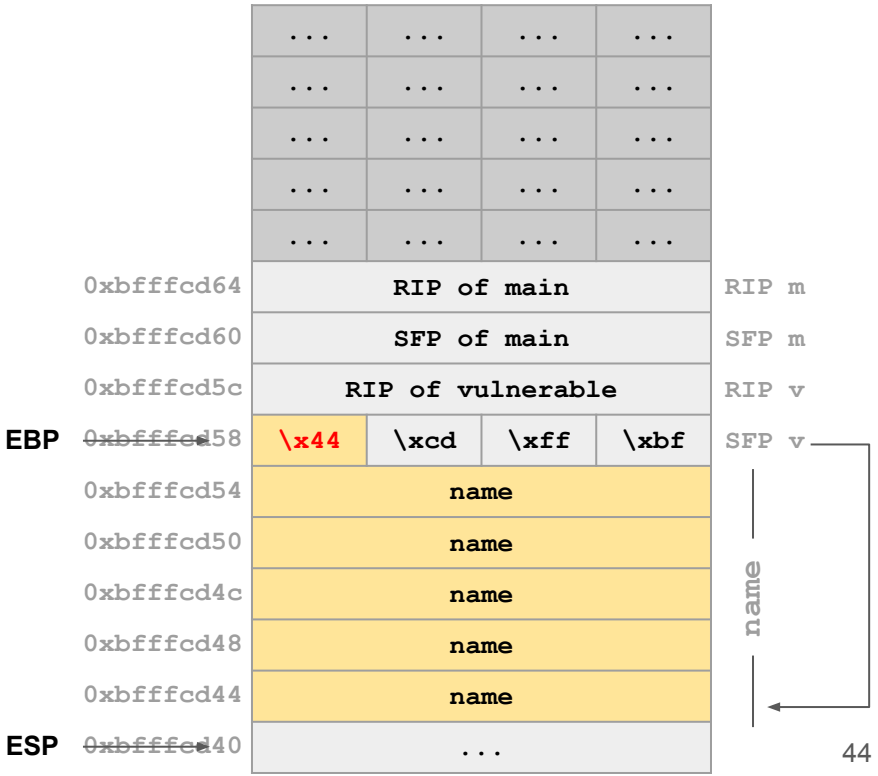
```
void vulnerable(void) {
    char name[20];
    fread(name,21,1,stdin);
}

int main(void) {
    vulnerable();
    return 0;
}
```

```
vulnerable:
    ...
    call gets
    add $4, %esp
    mov %ebp, %esp
    pop %ebp
    ret

main:
    ...
    call vulnerable
    mov %ebp, %esp
    pop %ebp
    ret
```

EIP →



Off-by-one

The C program now thinks that the SFP of **`main`** and the RIP of **`main`** are inside **`name`**.

The attacker controls these values, so the attacker can now overwrite where the program thinks the RIP of **`main`** is.

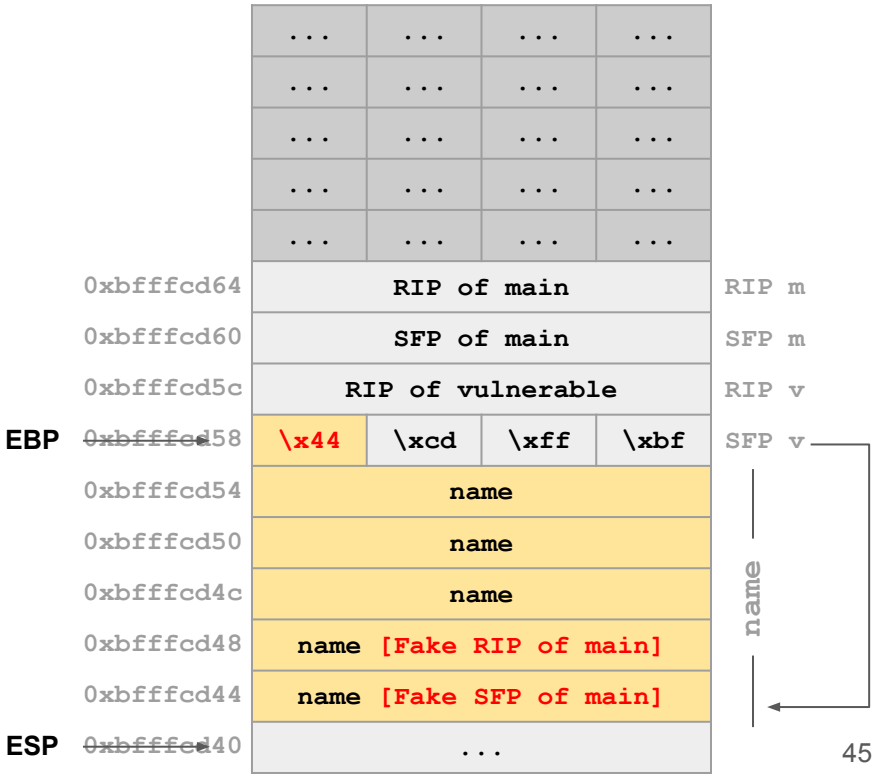
```
void vulnerable(void) {
    char name[20];
    fread(name,21,1,stdin);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call gets
    add $4, %esp
    mov %ebp, %esp
    pop %ebp
    ret

main:
    ...
    call vulnerable
    mov %ebp, %esp
    pop %ebp
    ret
```



Off-by-one

Let's see what happens when the **`vulnerable`** function returns.

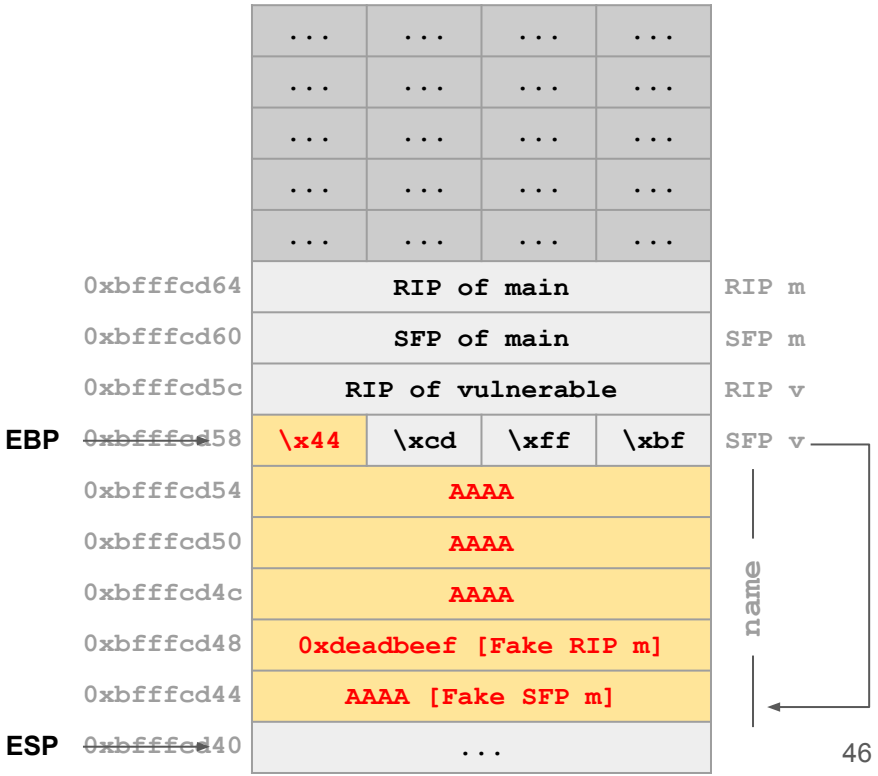
```
void vulnerable(void) {
    char name[20];
    fread(name,21,1,stdin);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call fread
    add $16, %esp
    mov %ebp, %esp
    pop %ebp
    ret

main:
    ...
    call vulnerable
    mov %ebp, %esp
    pop %ebp
    ret
```



Off-by-one

Let's see what happens when the **`vulnerable`** function returns.

Returned from **`fread`**, preparing to return from **`vulnerable`**.

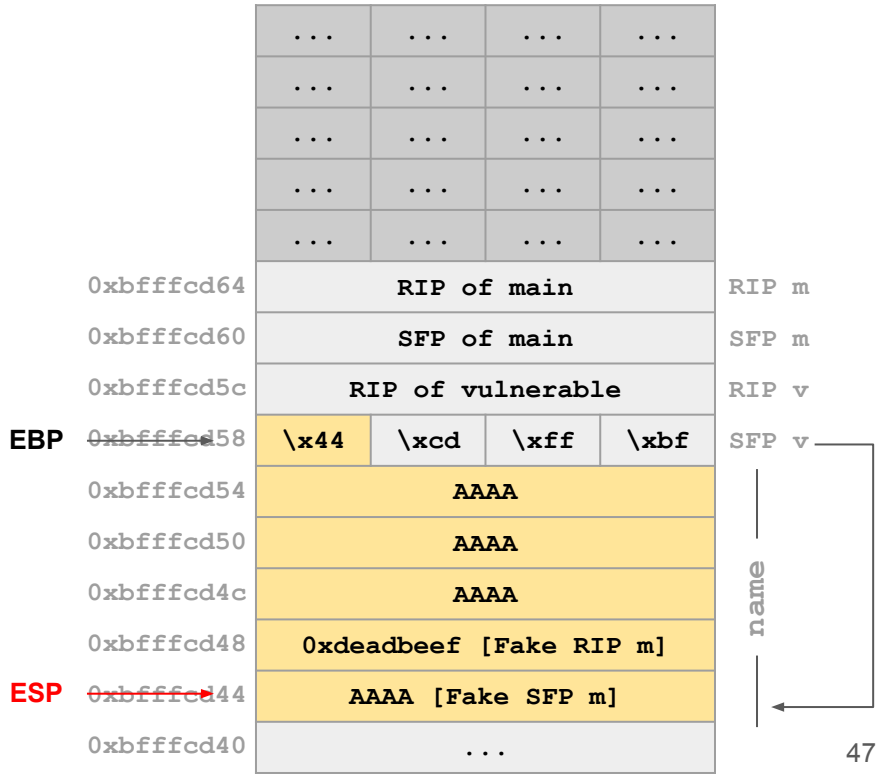
```
void vulnerable(void) {
    char name[20];
    fread(name,21,1,stdin);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call fread
    add $16, %esp
    mov %ebp, %esp
    pop %ebp
    ret

main:
    ...
    call vulnerable
    mov %ebp, %esp
    pop %ebp
    ret
```



Off-by-one

Let's see what happens when the **`vulnerable`** function returns.

Epilogue step 1: Move ESP back up.

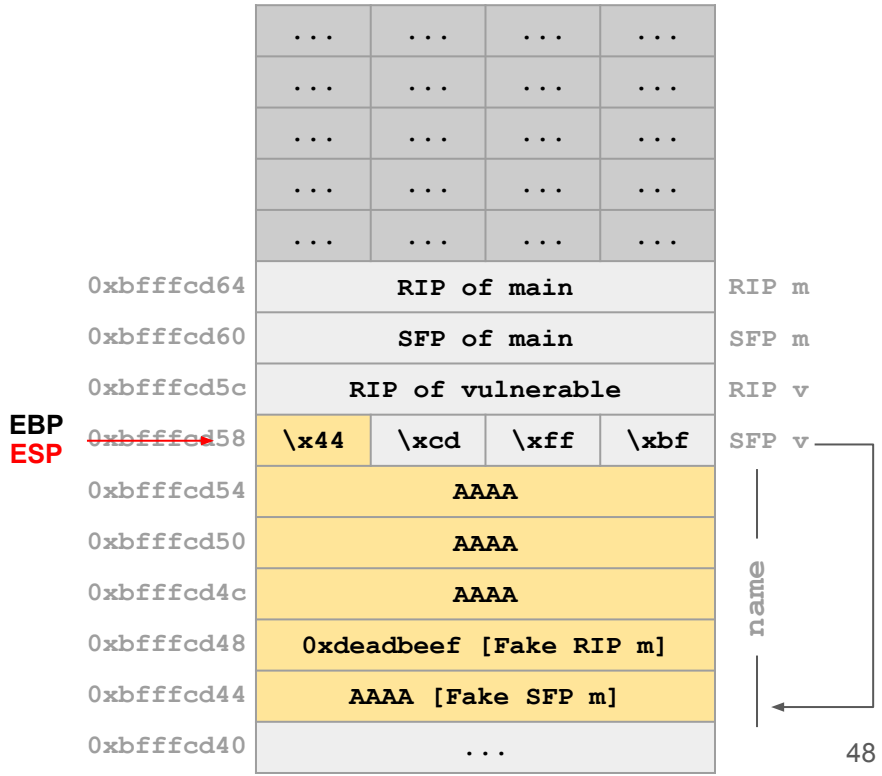
```
void vulnerable(void) {
    char name[20];
    fread(name,21,1,stdin);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call fread
    add $16, %esp
    mov %ebp, %esp
    pop %ebp
    ret

main:
    ...
    call vulnerable
    mov %ebp, %esp
    pop %ebp
    ret
```



Off-by-one

Let's see what happens when the **`vulnerable`** function returns.

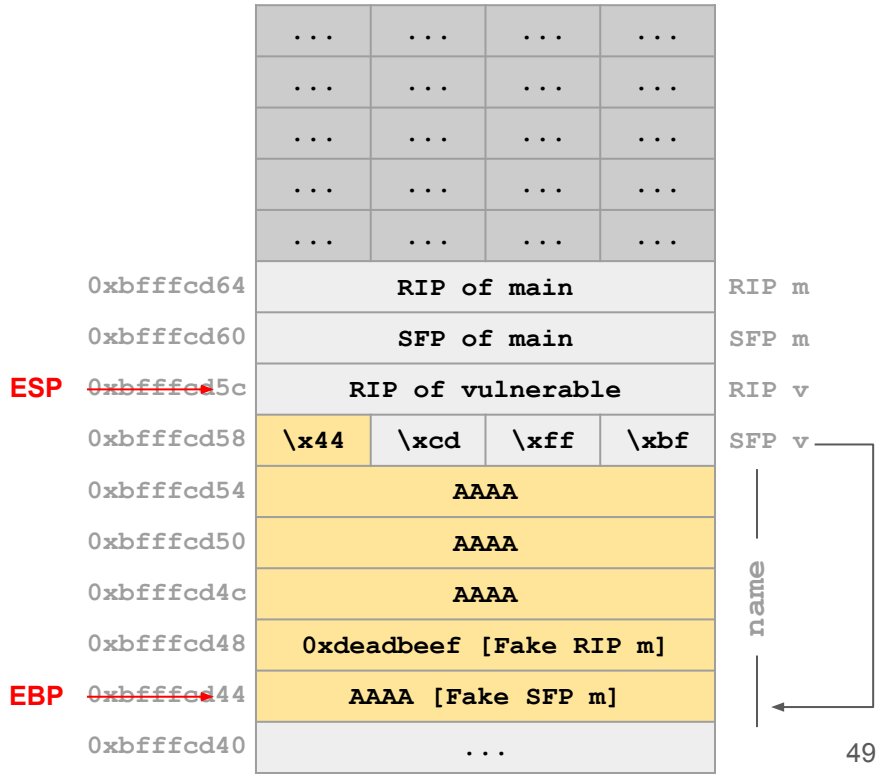
Epilogue step 2: Restore EBP. Note that EBP now points inside **`name`**, instead of at the SFP of **`main`**.

```
void vulnerable(void) {
    char name[20];
    fread(name,21,1,stdin);
}

int main(void) {
    vulnerable();
    return 0;
}
```

```
vulnerable:
    ...
    call fread
    add $16, %esp
    mov %ebp, %esp
    pop %ebp
EIP → ret

main:
    ...
    call vulnerable
    mov %ebp, %esp
    pop %ebp
    ret
```



Off-by-one

Let's see what happens when the **`vulnerable`** function returns.

Epilogue step 3: Restore EIP. We never changed the RIP of **`vulnerable`**, so execution returns to **`main`** as normal.

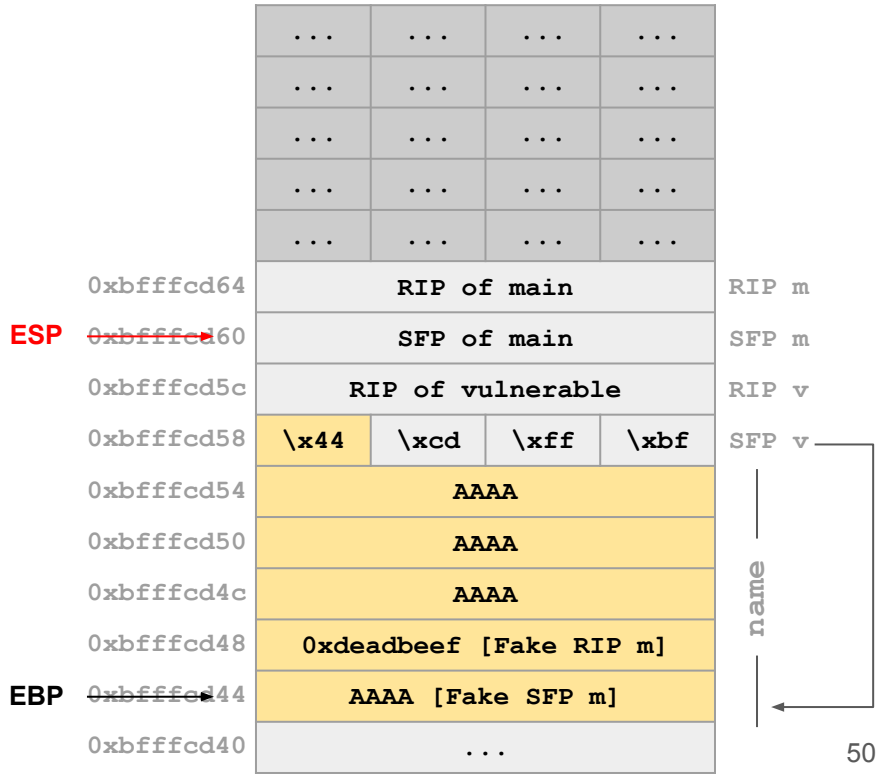
```
void vulnerable(void) {
    char name[20];
    fread(name,21,1,stdin);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call fread
    add $16, %esp
    mov %ebp, %esp
    pop %ebp
    ret

main:
    ...
    call vulnerable
    mov %ebp, %esp
    pop %ebp
    ret
```



Off-by-one

Let's see what happens when the **`main`** function returns, now with the EBP in the wrong place.

Epilogue step 1: Move ESP back up.

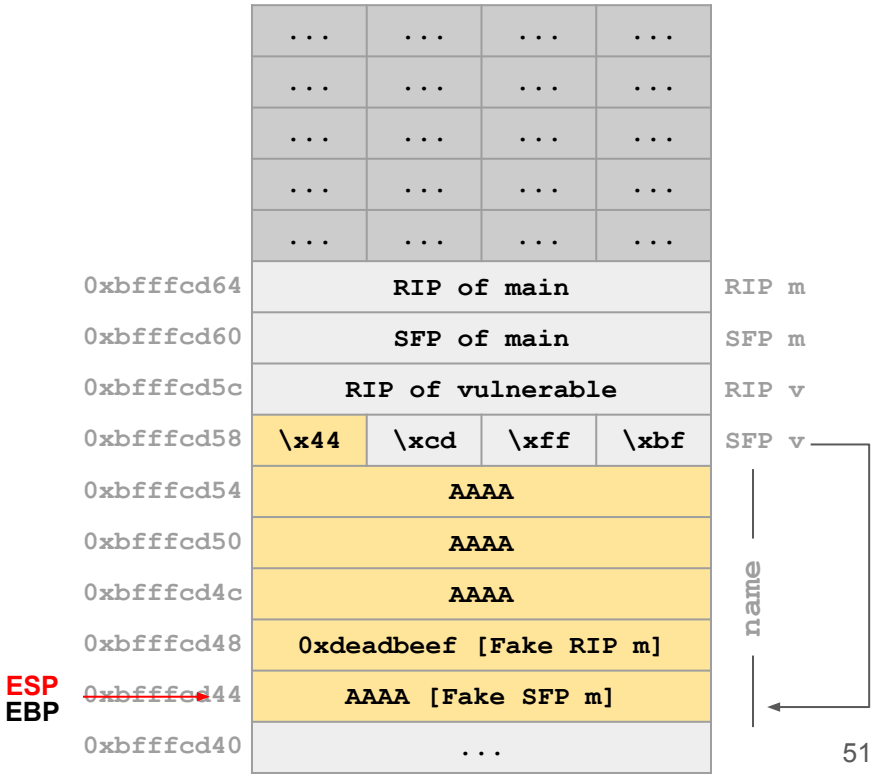
```
void vulnerable(void) {
    char name[20];
    fread(name,21,1,stdin);
}

int main(void) {
    vulnerable();
    return 0;
}
```

```
vulnerable:
    ...
    call fread
    add $16, %esp
    mov %ebp, %esp
    pop %ebp
    ret

main:
    ...
    call vulnerable
    mov %ebp, %esp
    pop %ebp
    ret
```

EIP →



Off-by-one



Let's see what happens when the **`main`** function returns, now with the EBP in the wrong place.

Epilogue step 2: Restore EBP. The program looks at our fake SFP to restore EBP, and points EBP to garbage AAAA.

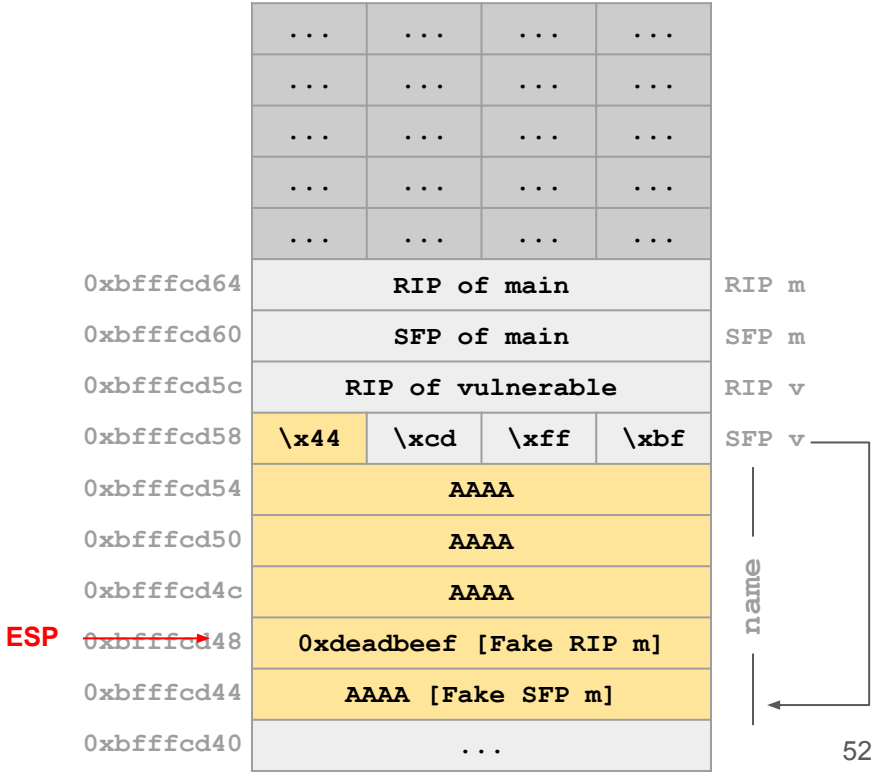
```
void vulnerable(void) {
    char name[20];
    fread(name,21,1,stdin);
}

int main(void) {
    vulnerable();
    return 0;
}
```

```
vulnerable:
    ...
    call fread
    add $16, %esp
    mov %ebp, %esp
    pop %ebp
    ret

main:
    ...
    call vulnerable
    mov %ebp, %esp
    pop %ebp
    ret
```

EIP →



Off-by-one



Let's see what happens when the **`main`** function returns, now with the EBP in the wrong place.

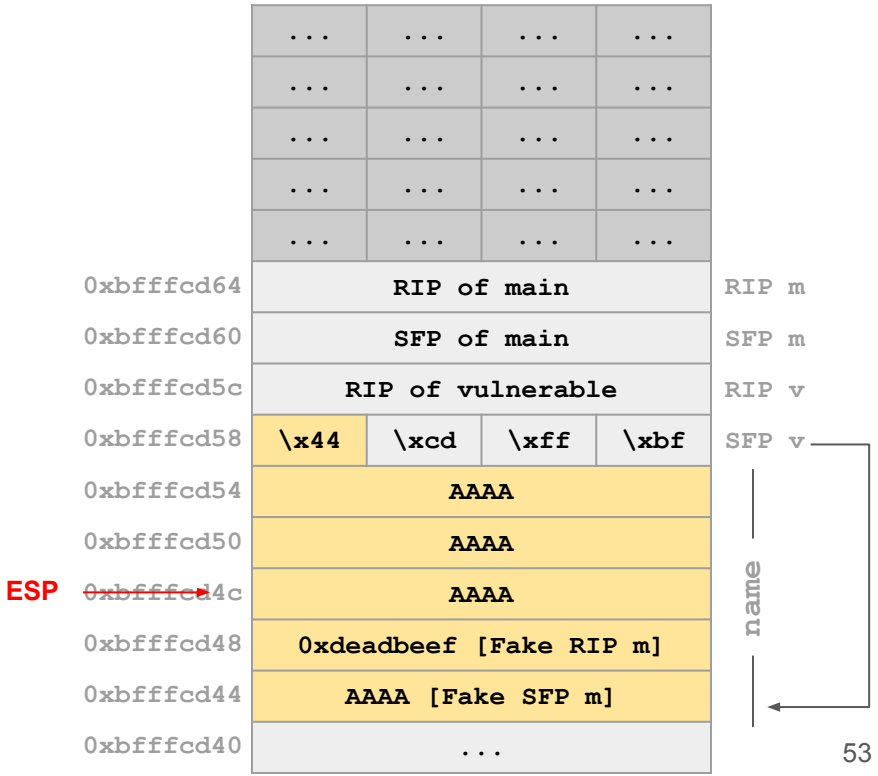
Epilogue step 3: Restore EIP. The program looks at our fake RIP to restore EIP, and redirects execution to **`0xdeadbeef`**.

```
void vulnerable(void) {
    char name[20];
    fread(name,21,1,stdin);
}

int main(void) {
    vulnerable();
    return 0;
}
```

```
vulnerable:
    ...
    call fread
    add $16, %esp
    mov %ebp, %esp
    pop %ebp
    ret

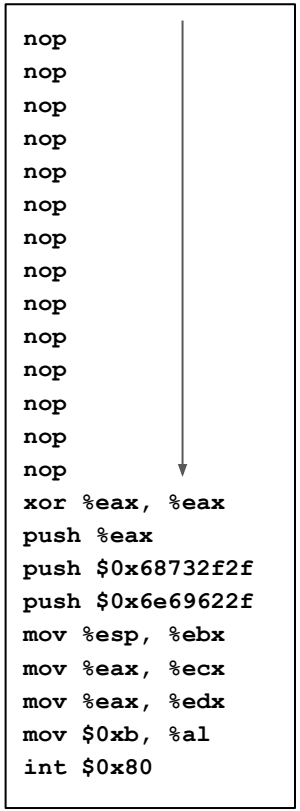
main:
    ...
    call vulnerable
    mov %ebp, %esp
    pop %ebp
    ret
```



Writing Robust Exploits

NOP Sleds

- Idea: Instead of having to jump to an exact address, make it “close enough” so that small shifts don’t break your exploit
- **NOP**: Short for no-operation or no-op, an instruction that does nothing (except advance the EIP)
  - A real instruction in x86, unlike RISC-V
- Chaining a long sequence of NOPs means that landing anywhere in the sled will bring you to your shellcode



Summary: Memory Safety Vulnerabilities

- **Format string vulnerabilities**: An attacker exploits the arguments to printf
- **Heap vulnerabilities**: An attacker exploits the heap layout
- **Writing robust exploits**: Making exploits work in different environments

- Memory safety vulnerabilities have a high impact: If your code has a memory safety vulnerability, the attacker can take complete control of your program

- Next: Defending against memory safety vulnerabilities